
```
>>> printsquare(square)
```

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	22	47	16	41	10	35	4
[1]	5	23	48	17	42	11	29
[2]	30	6	24	0	18	36	12
[3]	13	31	7	25	43	19	37
[4]	38	14	32	1	26	44	20
[5]	21	39	8	33	2	27	45
[6]	46	15	40	9	34	3	28

There is just one final change we need to make. Because of the rules of the % operator, our square contains consecutive integers between 0 and 48, but Kurushima's algorithm is meant to fill our square with the integers from 1 to 49. We can add one line that replaces 0 with 49 in our square:

```
square=[[n**2 if x == 0 else x for x in row] for row in square]
```

Now our square is complete. We can verify that it is indeed a magic square by using the `verifysquare()` function we created earlier:

```
verifysquare(square)
```

This should return `True`, indicating that we've succeeded.

We just created a 7×7 magic square by following Kurushima's algorithm. Let's test our code and see if it can create a larger magic square. If we change `n` to 11 or any other odd number, we can run exactly the same code and get a magic square of any size:

```
n = 11
square=[[float('nan') for i in range(0,n)] for j in range(0,n)]

center_i = math.floor(n/2)
center_j = math.floor(n/2)

square[center_i][center_j] = int((n**2 + 1)/2)
square[center_i + 1][center_j] = 1
square[center_i - 1][center_j] = n**2
square[center_i][center_j + 1] = n**2 + 1 - n
square[center_i][center_j - 1] = n
```
